ABSTRACT

This project provides a comprehensive comparative analysis of two widely used path-planning algorithms: A* (A-star) and RRT (Rapidly-exploring Random Tree), focusing on their applications in autonomous drone navigation. The study involves implementing both algorithms in MATLAB to create 3D path-planning simulations within a dynamically structured environment filled with obstacles, similar to those encountered in real-world search and rescue or inspection scenarios.

A* is a heuristic-based search algorithm known for finding optimal paths by expanding nodes systematically, while RRT is a sampling-based method designed for the rapid exploration of high-dimensional spaces. Each algorithm's performance was evaluated based on key metrics, including path efficiency (measured by path length and smoothness), computational complexity (execution time and memory usage), and obstacle-handling capabilities. The findings highlight significant contrasts in the suitability of each approach: A* excels in structured, grid-based environments with fewer dimensional constraints, providing optimal paths, while RRT is better suited to complex, high-dimensional spaces where rapid, feasible path generation is prioritized over strict optimality.

This analysis offers valuable insights into the strengths and limitations of A* and RRT for diverse autonomous navigation tasks, providing a foundation for selecting appropriate path-planning strategies in specific operational contexts.

Keywords: A*, RRT

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work. It's a matter of pride and privilege to express my deep gratitude to the HITS management for providing me with the necessary facilities and support.

We are highly elated to express my sincere and abundant respect to the Vice-Chancellor,**Dr. S. N. Sridhara**, for giving me this opportunity to present and implement my ideas in this project. We wish to express my heartfelt gratitude to**Dr. S Janaki Raman,** Associate Professor and HoD i/c, Department of Mechatronics Engineering, for his valuable support and encouragement in carrying out this work.

We would like to thank my guide, **Dr. S Janaki Raman,** Associate Professor of the Department of Mechatronics Engineering for continually guiding and mentoring us by giving valuable suggestions to complete the project work.

We are exceptionally obligated to our project coordinators, **Dr. S Janaki Raman** and **Sivakamasudari** for their direction and steady supervision throughout our project. We would like to thank all the technical and teaching staff, and the Department of Mechatronics Engineering for all their support.

Last, but not least, my thanks and thanks additionally go to my partner in building up the venture and individuals who have energetically gotten me out with their capacities. We are deeply indebted to my parents who have been the greatest support throughout the project.

Shaik Sameer Salahudduin **22125004**

Thirisha.R **22125022**

Adam Hamdullah Muneer Vm **22125028**

HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEER

EMD51512-AI&ROBOTICS

Project Batch No	**:**	10
Title of The Project	**:**	PERSONAL ASSISTIVE ROBOT
Number of The Students In The Batch	**:**	3

Student'sIndividualContribution

Nameof the Student: Shaik Sameer Salahuddin

Register No.:22125004

Contribution content is reported on pages: A literature review,Developed and implemented A* For the robot, Voice recognition module programming, power circuit design, Robotic arm kinematics, Python programming

I have gone through relevant research papers as part of a literature review and contributed to the development of an autonomous robot. I developed and implemented the A* algorithm for the robot, programmed the voice recognition module, and worked on power circuit design. I also handled robotic arm kinematics and contributed to system integration through Python programming.

Shaik Sameer Salahuddin (22125004)

HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEERING

EMD51512-AI&ROBOTICS

Project Batch No	**:**	10
Title of the project	**:**	PERSONAL ASSISTIVE ROBOT
Number of the students in the batch	**:**	3

Student'sIndividualContribution

Nameof the Student: Thirisha. R

Register No.:22125022

Contribution content is reported on pages: A literature review,Object detection and face recognition using YOLO11n, CAD modelling of the Robot, Robotic arm kinematics, Components selection, Hardware integration, Raspberry pi programming

I have done a literature review ofresearch papers to support the development of an autonomous robotic system. I have implemented object detection and face recognition using YOLOv11n, designed the robot's CAD model, and developed robotic arm kinematics. I have also handled component selection, hardware integration, and Raspberry Pi programming to ensure seamless system functionality.

Thirisha.R (22125022)

HINDUSTAN
INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF MECHATRONICS ENGINEERING

EMD51512-AI&ROBOTICS

Project Batch No	**:**	10
Title of the project	**:**	PERSONAL ASSISTIVE ROBOT
Number of the students in the batch	**:**	3

Student'sContribution

Nameof the Student: Adam Hamdullah Muneer Vm

Register No.:22125028

Contribution content is reported on pages: A literature review, Voice recognition module training , Components selection, Components Setup, Hardware integration,

I have gone through research papers as part of a literature review to support the development of the system. I trained the voice recognition module, carried out component selection and setup, and contributed to hardware integration.

Adam Hamdullah Muneer Vm(22125028)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

With the rapid development of technology, smart robots are slowly becoming a part of our daily lives. These robots are not just seen in factories or research labs anymore — they are now being designed to help people in homes, schools, and even public places. This project is about creating a Personal Assistive Robot — a smart and interactive robot that can help users with simple everyday tasks by understanding voice commands, detecting objects, recognizing faces, and picking and placing small items.

The main idea of this robot is to act like a helpful assistant at home. It can do tasks like picking up something from the floor, bringing a small object to the user, or simply responding to commands. It is not made for a specific group of people, like the elderly or people with disabilities. Instead, it is a general-purpose robot that can be useful for anyone. It can also be a companion that interacts and communicates with users, making technology more friendly and accessible.

To make this robot smart, **a** Raspberry Pi 5 is used as the main controller. This small but powerful device takes care of all the processing work. A NOIR Pi Camera is attached to the robot to help it see the surroundings. The camera captures real-time images and videos, which are then processed using the YOLOv11n object detection algorithm. With this, the robot can detect and identify different objects and human faces. This helps the robot decide what actions to take — for example, recognizing a person and responding to them or identifying an object and moving towards it.

To allow users to interact with the robot, a voice recognition system is used. The robot listens to the user's commands, understands them, and then gives a response using a text-to-speech system. This means users can simply talk to the robot and get a voice reply or see it perform the requested action.

The robot is also built with two robotic arms, one on each side. Each arm has 5 degrees of freedom (5-DOF**)**, which means they can move in multiple directions and perform different actions. These arms are equipped with grippers that can hold and move objects. For movement, the robot uses two motorized wheels, allowing it to move smoothly in indoor spaces. It can go toward the object it detects, stop at the right position, and use its arms to interact with the object.

In short, this robot is a combination of vision, voice, and physical interaction. It brings together hardware and software in a smart and useful way. The purpose of the project is to build a robot that is not only functional but also interactive and easy to use in real-life situations. It shows how technologies like AI, computer vision, and robotics can work together to create a simple, helpful assistant for home environments.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction:

The integration of robotics into daily human life is rapidly evolving, with a strong focus on creating systems that go beyond mechanical assistance to provide meaningful social and emotional interaction. As robots transition from industrial tools to companions and assistants in domestic, healthcare, and collaborative work environments, the need for socially intelligent and emotionally responsive systems becomes increasingly critical. Socially Interactive Robots (SIRs) and assistive robots are designed to bridge this gap by mimicking human communication cues and adapting to user behavior, thereby enhancing usability, trust, and long-term acceptance.

Recent advancements in artificial intelligence, cognitive science, and human-robot interaction have laid the groundwork for robots that not only perform tasks efficiently but also understand and respond to human emotions, intentions, and social contexts. These developments hold promise for a wide range of applications, from eldercare and disability support to personalized healthcare and industrial collaboration. However, despite the promising outcomes, challenges remain in achieving natural, context-aware interaction and sustained user engagement over time.

This chapter explores existing research in socially interactive and assistive robotics, providing a literature-based foundation to understand the current state of the field. It examines various approaches to emotional modeling, interaction modalities, intention recognition, and design principles that shape human-robot relationships. Through a critical review and synthesis of prior work, the chapter also identifies existing limitations and uncovers gaps in the research that must be addressed to realize the vision of truly empathetic and adaptive robotic systems.

2.2. Literature Review

Ishiguro et al. [1]

This paper presents the development of "Robovie," an interactive humanoid robot designed to foster meaningful social interactions and build long-term relationships with humans. The robot’s architecture is inspired by findings from cognitive experiments, where human interaction patterns and emotional cues were analyzed. Robovie features multimodal communication capabilities, such as speech, gestures, and gaze control, to enable natural interactions. The researchers emphasize that combining cognitive science with robotics can lead to emotionally aware and socially capable robots.

Inference from Research Paper:

Integrating cognitive models into robotic systems enhances their social presence and acceptance. The work underscores the necessity of interdisciplinary collaboration to create robots that can understand and reciprocate human emotions.

Fong et al. [2]

This survey paper offers a broad overview of socially interactive robots (SIRs), establishing a framework to understand how these systems are designed and evaluated. It introduces a detailed taxonomy that includes robot roles (e.g., peer, assistant, mediator), interaction modalities (verbal, non-verbal), and system components (sensors, actuators, control architecture). Additionally, it assesses the psychological and societal implications of deploying such systems in public and private spaces.

Inference from Research Paper:

The taxonomy serves as a foundation for future research by categorizing the complexities of SIR design. The paper draws attention to the emotional and social dimensions of human-robot interaction, pushing the boundaries beyond functional performance.

Ratsamee et al. [3]

This paper discusses the design and deployment of a “Robot Companion” tailored for Industry 5.0, with a focus on human intention recognition and cooperative task execution. By analyzing human path prediction and face orientation, the robot anticipates human movement to assist in industrial settings. It showcases the integration of vision systems, machine learning, and embedded control for dynamic assembly line collaboration.

Inference from Research Paper:

The study proves that robots capable of interpreting human intention can significantly enhance safety and productivity in shared workspaces. Human-aware navigation and gesture-based interaction are pivotal for smooth cooperation in industrial contexts.

Mišeikis et al. [4]

This work introduces "Lio," a mobile service robot engineered for caregiving and social assistance in healthcare institutions. Lio is equipped with a sensor suite including LiDAR, cameras, and tactile feedback systems, alongside a robotic arm capable of grasping objects and interacting with patients. Its design emphasizes privacy protection, patient engagement, and low cognitive load for users. Lio's

autonomous and semi-autonomous functionalities range from logistical support (e.g., item delivery) to patient engagement through conversation and reminders.

Inference from Research Paper:
Robots like Lio demonstrate how assistive technologies can bridge staffing gaps in healthcare. A user-friendly design and context-aware autonomy are essential for real-world deployment in sensitive environments like hospitals and care homes.

Mariagrazia Costanzo et al. [5]
This paper reviews the current state of AI-driven assistive technologies for the elderly, including smart home systems, wearable health monitors, and interactive robots. The review focuses on the psychological acceptance of these technologies, especially their perceived usefulness, ease of use, and emotional impact. It addresses how socio-cultural factors and caregiver involvement influence the success of these systems.

Inference from Research Paper:
While technology plays a crucial role in eldercare, human factors remain paramount. Acceptance depends not only on functionality but also on how empathetically and respectfully the technology integrates into daily life.

Duerstock et al. [6]
This report evaluates the deployment of assistive robots for people with disabilities, examining how mobile platforms can aid in daily tasks, environmental navigation, and communication. The focus is on enhancing independence and reducing caregiver strain. Robots in this domain must navigate diverse environments and interact through accessible user interfaces such as voice commands and adaptive touchscreens.

Inference from Research Paper:
For persons with disabilities, robots must be intuitive, highly reliable, and customizable. The study highlights the potential of robotics to improve autonomy and inclusion for differently-abled individuals, provided the systems are ethically designed and context-aware.

Akkas U. Haque [7]

This research presents a robotic arm capable of being controlled by spoken language commands. It features five degrees of freedom and utilizes a PIC18f452 microcontroller. The voice command

system is developed using Microsoft Visual Studio to process spoken input via a microphone. Key functionalities include:Movement in various directions,Gripping and releasing,Rotational control.This system is cost-effective and aimed at environments where manual operation may be unsafe or inconvenient (e.g., industrial automation, assistive tech).

Inference from Research Paper:

This paper provides a strong foundation for integrating voice command interfaces with mechanical motion control.Shows how simple microcontrollers (like PIC18f452 or Arduino) can handle real-time tasks in response to voice input.You can replicate or enhance their voice-to-action pipeline in your robot to control arm gestures or navigation.

Shakil Rashed [8]

This paper describes a voice-activated robotic arm that operates wirelessly using an Arduino UNO, NRF wireless module, and a voice recognition module. The arm can execute basic commands like “move up/down”, “grab”, etc.

Inference from Research Paper:

Suggests a modular, wireless setup ideal for mobile assistive robots.Reinforces the efficiency of using limited, clear voice commands.Helps you consider feedback loops for command acknowledgment (e.g., LED or audio confirmation).

Uriel Martinez-Hernandez, Benjamin Metcalfe[9]

A comprehensive review of wearable assistive robots that support people with mobility limitations. Topics covered include:Adaptive control using machine learningImportance of comfort and human-machine interaction (HMI)Real-world challenges in implementation (e.g., power, cost, ergonomics)

Inference from Research Paper:

Emphasizes need for user adaptability and intuitive interfaces—speech and vision are key here.Encourages designing a robot that learns from interaction history.Pushes the idea that robots should adjust to different user profiles (elderly, disabled, etc.).

Samuel bakuto[10]

Covers a consumer-level wearable robotic exoskeleton for athletes and casual users. It features assistance modes for walking/running, designed for **ergonomic support**, **mobility enhancement**, and **user comfort.**

Inference from Research Paper:

Highlights how assistive robots must be lightweight, practical, and easy to use.Suggests implementing multiple operation modes in your robot—such as “assist mode,” “rest mode,” and “interactive mode.”Inspires a human-centered design philosophy: the robot should feel like a helpful companion, not a machine.

Shaikh Khaled Mostaque [11]

This study presents a cost-effective robotic arm system controlled via voice commands transmitted through a smartphone's Bluetooth interface. The system utilizes an Arduino Mega microcontroller and incorporates sonar sensors for object detection, enabling the robot to autonomously detect and pick up objects upon receiving voice instructions.

Inference from Research Paper:

The integration of voice control with object detection in this paper aligns closely with our project's objectives. The use of readily available components like Arduino and Bluetooth modules demonstrates a practical approach to developing assistive robots. Inspiration can be drawn from their methodology to enhance the autonomy and user-friendliness of our robot.

Vishnu Prabhu S [12]

This paper discusses the development of a robotic arm controlled by voice commands, capable of detecting and classifying objects. The system employs Arduino for control, Processing software for speech recognition, and Roborealm for image processing, enabling the robot to perform tasks like picking up and sorting objects based on their classification.

Inference from Research Paper:

The combination of voice control with object detection and classification offers a comprehensive approach to assistive robotics. Implementing a similar multi-software integration can enhance your robot's functionality, allowing it to interact more intelligently with its environment and perform complex tasks autonomously.

Z. Wang [13]

This research introduces a six-degree-of-freedom robotic arm that integrates voice control via Amazon Alexa, ultrasonic sensors for distance measurement, and computer vision for object recognition. The system calculates the position of objects using image processing and adjusts the arm's movements accordingly to grasp objects accurately.

Inference from Research Paper:

Incorporating voice assistants like Alexa can provide a more natural user interface for your robot. Additionally, combining ultrasonic sensors with computer vision enhances object detection accuracy. Adopting similar technologies can improve your robot's interaction capabilities and precision in performing assistive tasks.

Adekunle Taofeek Oyelami [14]

This paper presents a lightweight, voice-controlled robotic arm with four degrees of freedom, designed to assist individuals with impairments. The system utilizes an Arduino microcontroller and a speech recognition module to interpret voice commands, enabling the arm to perform tasks like grasping and moving objects.

Inference from Research Paper:

The focus on assistive applications in this study aligns with our project's goals. Implementing a similar design can provide a foundation for developing a user-friendly, voice-controlled robotic arm tailored to assist individuals with mobility challenges.

Sika K[15]

This research focuses on interactive control of a robotic arm using voice recognition, providing a user-friendly approach to controlling robotic systems through voice commands. The system is built on Arduino, integrating key components such as servo motors for precise movements, Bluetooth for wireless communication, and motor drivers for higher torque applications. The HC-05 Bluetooth module supports voice recognition, enabling external devices to control the robotic arm through voice commands. This setup allows for quick responses and minimal human intervention.

Inference from Research Paper:

Implementing a voice-controlled robotic arm using Arduino and Bluetooth can enhance the accessibility and ease of use of our assistive robot. The integration of voice recognition with wireless communication allows for remote control, making it suitable for users with mobility impairments.

Nicolas Viniegra [16]

This paper explores the control of a robotic hand through a combination of voice commands and biosignals. The system utilizes voice recognition to interpret user commands and biosignals to detect user intentions, enabling more intuitive control of the robotic hand. The integration of these modalities enhances the responsiveness and accuracy of the robotic system.

Incorporating biosignals alongside voice recognition can improve the accuracy and intuitiveness of our assistive robot's responses. This multimodal approach allows for more nuanced understanding of user intentions, enhancing the overall user experience.

K. Vibha [17]

This project outlines a vision-based, voice-commanded robotic system designed for handling dental tools, aiming to improve the quality and hygiene of dental services. Real-time computer vision is performed using OpenCV on a Raspberry Pi with a webcam, and voice recognition is achieved through the Google Speech API with a microphone. The robot identifies the required dental instruments and retrieves them upon voice command, minimizing physical contact and enhancing operational efficiency.

Inference from Research Paper:

The integration of computer vision with voice recognition in this study offers valuable insights for our assistive robot. Implementing similar technologies can enable our robot to identify and interact with objects based on visual and auditory cues, improving its functionality and user interaction

2.3 Summary and Research Gap:

The reviewed literature collectively emphasizes the growing importance of socially interactive and assistive robots across various domains, including healthcare, industrial collaboration, eldercare, and disability support. These robots integrate cognitive models, multimodal interaction, and human-intention recognition to improve user engagement, safety, and functionality. Studies such as those by Ishiguro et al. and Fong et al. highlight the value of emotional intelligence and taxonomy-based design frameworks, while works like those of Mišeikis et al. and Ratsamee et al. demonstrate practical applications in caregiving and industrial environments. Moreover, research by Costanzo and Duerstock underscores the psychological and societal dimensions of user acceptance and inclusivity.

2.4 Research Gap:

While current advancements showcase impressive progress in robot capabilities and social intelligence, there remains a noticeable gap in the seamless integration of emotional awareness, adaptive learning, and context-specific behavior across diverse real-world settings. Most systems are domain-specific and lack generalizability, limiting their broader applicability. Additionally, long-term user adaptation, ethical deployment, and cultural sensitivity are underexplored areas, calling for interdisciplinary research that combines AI, human psychology, and social sciences to develop universally acceptable, emotionally responsive robotic companions.

CHAPTER 3

METHODOLOGY

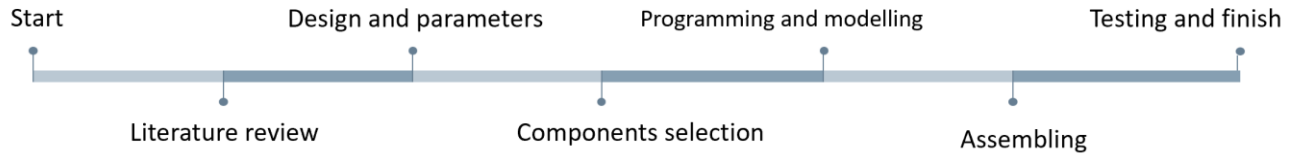
The literature review has been thoroughly completed, leading to the determination of several essential parameters for the robot. These parameters include a total weight of 5kg, ensuring the robot is lightweight and easily manoeuvrable. The lifting capacity is set at 2–3kg, providing ample strength for handling various objects and tasks. The inclusion of a 5DOF (Degrees of Freedom) robotic arm allows for versatile and precise movements, enhancing the robot's functionality. The motor operates at 100RPM (Revolutions Per Minute) with a torque of 98.1, ensuring smooth and powerful performance. Finally, the robot is equipped with wheels measuring 100mm in size, which contribute to its stability and mobility across different surfaces. These carefully considered specifications ensure that the robot is both efficient and capable of performing its intended tasks effectively.

As the next steps in the development process, a Computer-Aided Design (CAD) model of the robot will be created to provide a detailed visual representation and identify potential design issues. Following the CAD modelling, laser cutting technology will be used to fabricate most of the robot’s structural components using acrylic sheets, ensuring precision and durability. 3D printing will be employed specifically for the gripper and the charging station, where more complex and customized geometries are required. Once the components are prepared, appropriate parts and materials will be selected to ensure optimal performance. The programming of the robot's functions will then be undertaken, incorporating AI algorithms, voice interaction capabilities, and object detection mechanisms. Finally, rigorous testing will be conducted to evaluate the robot's performance, leading to necessary adjustments and improvements to achieve the desired

level of efficiency and reliability. Through this comprehensive development process, the robot will be refined and prepared for real-world applications.

3.1 Design and Parameter

The robot's motion is achieved through specific movements using servo motors. The base enables 360° yaw rotation with an MG995 servo, while the elbow joint, also powered by an MG995, allows limited pitch (up/down) movement. The wrist joint, controlled by an SG90 servo, provides 360° roll rotation, and the gripper, operated by another SG90, facilitates an open/close motion. The mechanical structure is built entirely using laser-cut acrylic sheets, which are precisely designed and assembled to support each joint and component. The base platform is formed from robust acrylic layers to house the yaw motor securely. The shoulder and elbow joints are created using interlocking acrylic frames that replicate standard robotic arm angles and support the servos. The upper arm consists of a



15 cm long, 50 mm wide acrylic segment, while the wrist joint is mounted onto a compact acrylic bracket designed for stability. The lower arm is constructed from a 5 cm long, 32 mm wide acrylic piece, providing a compact and lightweight frame for the gripper. The servos used include the SG90 with a torque of 1.8 kg.cm and the MG995 with a torque of 10 kg.cm, ensuring adequate strength for smooth and controlled movements.

The charging station and robot power system consist of essential components for efficient battery management and power delivery. The charging station includes a 12V-5A power adapter, a DC-DC step-down converter (XL4015 with a voltmeter) for voltage regulation, and copper strips or tape for contact-based charging. It also features Schottky diodes (10A, 1N5822) to prevent reverse current, a 10A fuse for protection, and a 12V charging indicator LED with a resistor. A DC barrel jack connector, selected based on the adapter, ensures a secure power connection.

For the robot's power and battery charging connections, a 3S BMS module (12.6V, 25A) for 18650 batteries manages charging and discharging. The system is powered by a LiPo battery, with connectors such as JST or XT60 for secure wiring. Epoxy glue or a hot glue gun ensures durable mounting and insulation.

Additionally, the robot incorporates a Raspberry Pi 5 as its central controller, with a Raspberry Pi Camera Noir Module 3 for vision-based applications. An AI Think Voice Recognition Module enables voice command functionality, while an ultrasonic sensor assists in distance measurement. The L298N motor driver controls motor movement, a 16-bit servo driver manages multiple servos efficiently, and 200 RPM motors provide mobility and speed for the robot.

Main functionality

- Perform pick-and-place tasks with its gripper.
- Rotate and orient objects using wrist and elbow joints.
- Navigate spaces with obstacle detection.
- Interact with users via voice commands and camera-based recognition.
- Recharge autonomously using a contact-based smart charging dock.

3.2 COMPONENTS SELECTION

3.2.1 MOTOR AND WHEELS SELECTION

To determine the torque and RPM required for a 10 kg robot designed to lift an additional 2–3 kg, several factors need to be considered, including the type of movement (e.g., wheeled or legged), the desired speed, and the lifting mechanism's design. Assuming a wheeled robot operating on flat terrain, each motor must generate enough torque to overcome the robot’s weight and any additional load while maintaining balance and control.

Torque and RPM Calculation forRobot

Parameters: (assumed)

1. **Total Weight**: 10 kg

2. **Acceleration**: 0.5 m/s²
3. **Desired Speed**: 1 m/s
4. **Wheel Diameters**: 70 mm and 100 mm

Formulas Used:

1. **Force (F)** = Mass (m) × Acceleration (a)

$$F = m \times aF = m \times a$$

2. **Torque (τ)** = Radius (r) × Force (F)

$$\tau = r \times F\tau = r \times F$$

Angular Velocity (ω) = $DesiredSpeed(\ v) \div Radius(\ r)\ \omega = vr\omega = \frac{v}{r}$

3. **RPM** = $\omega \times 602\pi \frac{\omega \times 60}{2\pi}$

Calculations:

For 70 mm Wheel Diameter:

- **Radius (r)** = 70 mm / 2 = 35 mm = 0.035 m
- **Force (F)** = 10 kg × 0.5 m/s² = 5 N
- **Torque (τ)** = 0.035 m × 5 N = **0.175 Nm**
- **Angular Velocity (ω)** = 1 m/s ÷ 0.035 m = 28.57 rad/s
- **RPM** = $28.57 \times 602\pi \frac{28.57 \times 60}{2\pi}$ = **272.84 RPM**

For 100 mm Wheel Diameter:

- **Radius (r)** = 100 mm / 2 = 50 mm = 0.05 m
- **Force (F)** = 10 kg × 0.5 m/s² = 5 N
- **Torque (τ)** = 0.05 m × 5 N = **0.25 Nm**
- **Angular Velocity (ω)** = 1 m/s ÷ 0.05 m = 20 rad/s
- **RPM** = $20 \times 602\pi \frac{20 \times 60}{2\pi}$ = **190.99 RPM**

Summary of Results:

Table- [1]

Wheel Diameter	Torque Required (Nm)	RPM Required
70 mm	0.175 Nm	272.84 RPM
100 mm	0.25 Nm	190.99 RPM

100 mm wheels were chosen to ensure better ground clearance, stability, and smooth mobility across various surfaces**.**

3.2.2 Robotic arm servo motor selection

Degrees of Freedom (DOF)

In robotics and mechanism design, the Degrees of Freedom (DOF) represent the number of independent parameters that define a system's configuration. For planar mechanisms, each rigid body (link) can move in 3 ways: two translational (X and Y directions) and one rotational.

To determine the DOF of a planar mechanism, we use Gruebler's Equation:

$DOF=3(N-1)-2C$

Where:

- NN = Total number of links, including the base.
- CC = Number of independent closed kinematic chains (loops).

Each closed loop introduces constraints that limit the system's mobility. As the number of closed chains increases, the overall freedom of the system decreases.

For a mechanism with:

- $N=4N$ = 4 links (including the base),
- $C=2C$ = 2 closed loops (independent constraints),

Apply the formula:$DOF=3(4-1)-2(2)=9-4=5$

Thus, the system has 5 degrees of freedom, meaning it can independently perform 5 unique movements. This DOF value indicates high flexibility and controllability, ideal for robotic arms, parallel linkages, or grippers requiring precise multi-axis control.

Table- [2]

Joint	Type of Motion	Motor Used	Rotation
Base	Yaw (360° Rotation)	MG995	360°
Shoulder	Pitch (Up/Down)	MG995	180°
Elbow	Pitch (Up/Down)	MG995	180°
Wrist	Roll (360° Rotation)	SG90	360°
Gripper	Open/Close	SG90	180°

Torque calculation

Joint	Torque Required	Servo Used	Servo Torque
Wrist (SG90)	0.85 kg.cm	SG90	1.8 kg.cm
Elbow (MG995)	2.7 kg.cm	MG995	10 kg.cm
Shoulder (MG995)	4.8 kg.cm	MG995	10 kg.cm
Base (MG995)	6.6 kg.cm	MG995	10 kg.cm
Gripper (SG90)	N/A	SG90	1.8 kg.cm

Kinematics Analysis

A. Forward Kinematics (FK)

$X = L1*\cos(\theta1) + L2*\cos(\theta1 + \theta2) + L3*\cos(\theta1 + \theta2 + \theta3)$

$Y = L1*\sin(\theta1) + L2*\sin(\theta1 + \theta2) + L3*\sin(\theta1 + \theta2 + \theta3)$

B. Inverse Kinematics (IK)

$\theta1 = \text{atan2}(Y, X)$

$\theta2 = \cos^{-1}((X^2 + Y^2 - L1^2 - L2^2) / (2 * L1 * L2))$

$\theta3 = \text{desired wrist orientation angle} - (\theta1 + \theta2)$

Force and Torque Calculator

	Mass (lbs)	Length (in)	Center of Mass		
Linkage 1	0.1212542	5.90551181	0.55115566		
Linkage 2	0.1212542	1.96850394	0.04409245		
Joint 2	1				
Object	1				
efficiency	65%				
Joint 1 Acc	360	Joint 2 Acc	180	deg/s^2	
	calculating:				
	Joint 1 Torque		1000.696	lbs/in	torque for first servo
	Joint 2 Torque		21.767	lbs/in	torque for second servo
servo	HS-255	has 26.878759 lbs/in			the servo I am using

Fig [1]- Force and Torque Calculator

Figure 1 depicts how forces are applied and how torque is calculated across the joints of the mechanism.

Forward Kinematics

For the forward kinematics section, enter in an angle for each joint, and the robot arm position will be displayed for you.

link	angle	x pt loc	y pt loc
J0		0	0
J1	150	-5.11	2.95
J2	35	-7.08	2.78

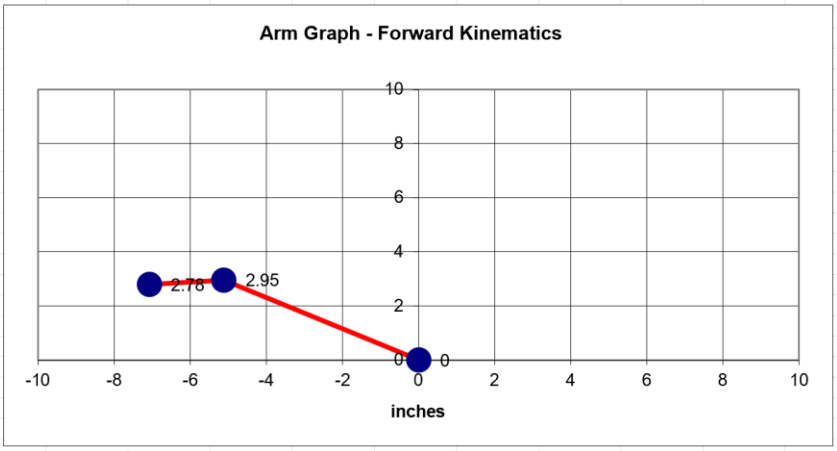


Fig [2] – Forward kinematics

Figure 2 illustrates the forward kinematics of the mechanism with joint angles J1 and J2 set to 150° and 35°, respectively.

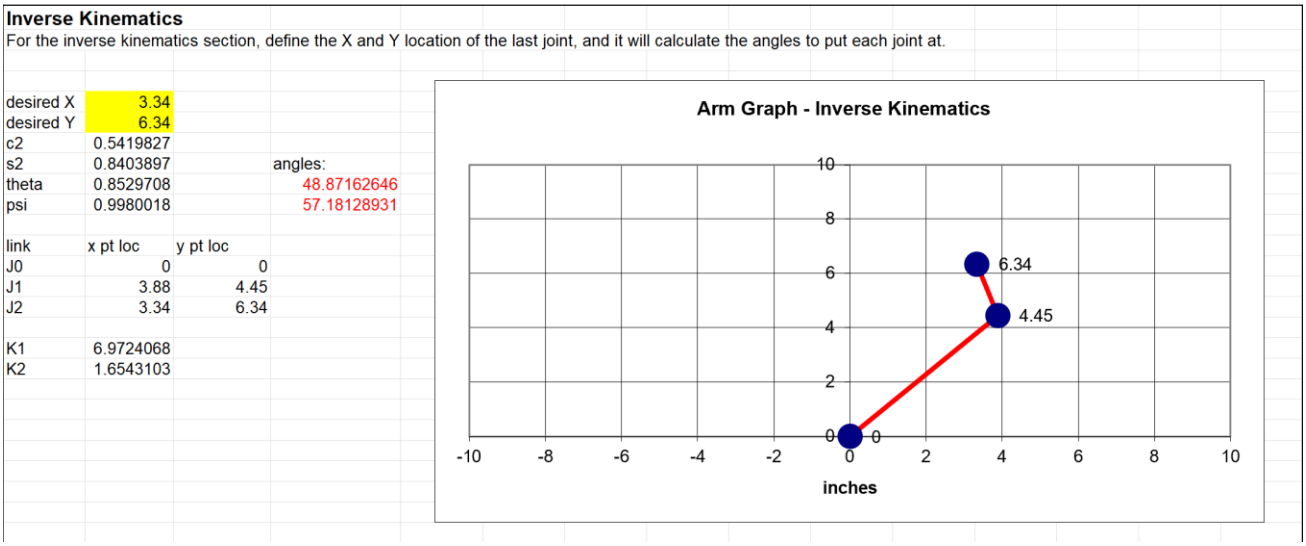


Fig [3]-Inverse kinematics

Figure 3 illustrates how the position of the end effector changes in response to variations in the X and Y coordinates.

3.2.3 Components descriptions

Raspberry pi 5 (4GB): The Raspberry Pi 5 serves as the central processing unit of the robot, featuring a quad-core 64-bit ARM Cortex-A76 processor running at 2.4 GHz. It comes with up to 8 GB of LPDDR4X RAM, dual micro-HDMI ports supporting 4K displays, USB 3.0 ports for fast data transfer, a PCIe interface, and a 40-pin GPIO header for hardware interfacing. These features make it powerful enough to handle vision processing, servo coordination, sensor integration, and AI applications simultaneously, making it an ideal platform for robotics and automation.

Figure [7]: Raspberry Pi 5

Camera Noir Module

The Camera Noir Module 3 is a 12MP camera with HDR support, autofocus, and the NoIR (No Infrared Filter) feature, allowing it to perform exceptionally well in low-light and IR-lit environments. Designed specifically for Raspberry Pi boards, this camera is ideal for computer vision tasks like face/object detection, tracking, and environmental awareness. The compact design and plug-and-play compatibility with the Raspberry Pi make it a reliable and efficient tool for vision-based robotic applications.

Figure [8]: Raspberry Pi Camera Noir Module 3

AI Think Voice Recognition Module

The AI Think Voice Recognition Module is a standalone, offline voice recognition system that supports customizable command input without requiring cloud connectivity. It processes voice commands locally, making it perfect for embedded systems like robots. With onboard microphones and a simple UART or GPIO interface, this module allows the robot to understand and respond to specific spoken commands, enabling hands-free operation and intuitive user interaction.

Figure [9]: AI Think Voice Recognition Module

Ultrasonic sensor

The ultrasonic sensor, typically an HC-SR04, enables distance measurement and object detection by emitting ultrasonic waves and calculating the time it takes for the echo to return. This sensor is crucial

for obstacle avoidance and safe navigation, allowing the robot to detect walls, objects, and users in its path. It offers reliable range detection from 2 cm to 400 cm and integrates easily with microcontrollers and SBCs like the Raspberry Pi.

Figure [10]: Ultrasonic Sensor

MG995

The MG995 is a high-torque servo motor capable of delivering up to 10 kg·cm of torque, making it suitable for lifting heavier loads and supporting critical joints like the robot's base and elbow. Operating at 4.8V to 6V, this servo provides powerful rotation and robust metal gear construction for durability. Its reliability under heavy mechanical load makes it a backbone component in robotic arm applications where strength and consistent positioning are essential.

SG90 Servo Motor

The SG90 is a lightweight, low-torque servo motor designed for precision control in low-load applications. With a torque of approximately 1.8 kg·cm and a wide operating angle, it's ideal for tasks like rotating the wrist or controlling a gripper. Its compact form factor and low power consumption make it perfect for smaller joints where agility and space efficiency are more critical than brute strength.

L298N Motor Driver

The L298N is a dual H-bridge motor driver capable of controlling two DC motors independently. It supports voltage ranges of 5V to 35V and can deliver up to 2A per channel, making it suitable for medium-load applications like mobile robots. The driver allows forward, reverse, and braking control via logic-level signals and integrates seamlessly with microcontrollers and single-board computers like the Raspberry Pi.

16-bit Servo Driver (PCA9685)

The PCA9685 16-channel, 12-bit PWM servo driver expands the number of controllable servos beyond the limitations of the Raspberry Pi's GPIO. Each channel can deliver precise control signals for individual servos, ensuring synchronized motion in multi-joint robotic systems. This driver communicates over I2C, allowing daisy-chaining and minimal pin usage for large-scale servo control.

200 RPM Motors

These DC motors operate at 200 revolutions per minute, providing a balance between torque and speed for mobile robotics. They are typically used in wheeled robots to ensure smooth, consistent movement. Their compatibility with standard motor drivers and gear attachments allows for easy integration and reliable ground navigation.

LiPo Battery

A Lithium Polymer (LiPo) battery provides high energy density, light weight, and stable voltage output, making it ideal for powering servo motors, sensors, and microcontrollers. The battery used typically has a 3S (11.1V or 12.6V when fully charged) configuration, supplying enough power for high-load robotics applications while maintaining portability and compact design.

3S BMS Module (12.6V, 25A)

The 3S Battery Management System is responsible for safely charging and discharging 3-cell LiPo battery packs. It protects against overcharging, over-discharging, overcurrent, and short circuits. With a 25A rating, this module ensures stable and safe power delivery to all robot components, extending battery life and preventing accidents.

18650 Batteries

18650 lithium-ion cells are commonly used in custom battery packs due to their high capacity, rechargeability, and compact cylindrical design. Each cell typically provides 3.7V and around 2000–3000mAh, making them suitable for building high-capacity power sources when arranged in series and parallel configurations for robotic applications.

JST / XT60 Connectors

JST and XT60 connectors are used for reliable, secure electrical connections between batteries, BMS modules, and power systems. JST connectors are compact and suitable for low-current signals, while XT60 connectors are rated for high current and are used in LiPo battery connections to ensure stable and safe power delivery under load.

12V-5A Power Adapter

The 12V-5A power adapter supplies stable

The components for the robot were selected based on detailed calculations of torque, degrees of freedom, and power requirements to ensure optimal performance. High-torque MG995 servos were chosen for the base and elbow joints to handle the load of the arm and payload, while SG90 servos were selected for the wrist and gripper due to their lower torque needs. A 5 DOF configuration was determined sufficient through mobility calculations, offering flexibility in manipulation. Power system components, including a 12V-5A adapter, XL4015 step-down converter, and a 3S BMS for LiPo batteries, were selected based on current and voltage demand estimates. The Raspberry Pi 5 was chosen as the central controller to manage complex processing, alongside a NoIR Camera Module 3 and AI Think Voice Module for vision and voice control, ensuring that each component aligns with the robot's mechanical and functional calculations.

Table [3]

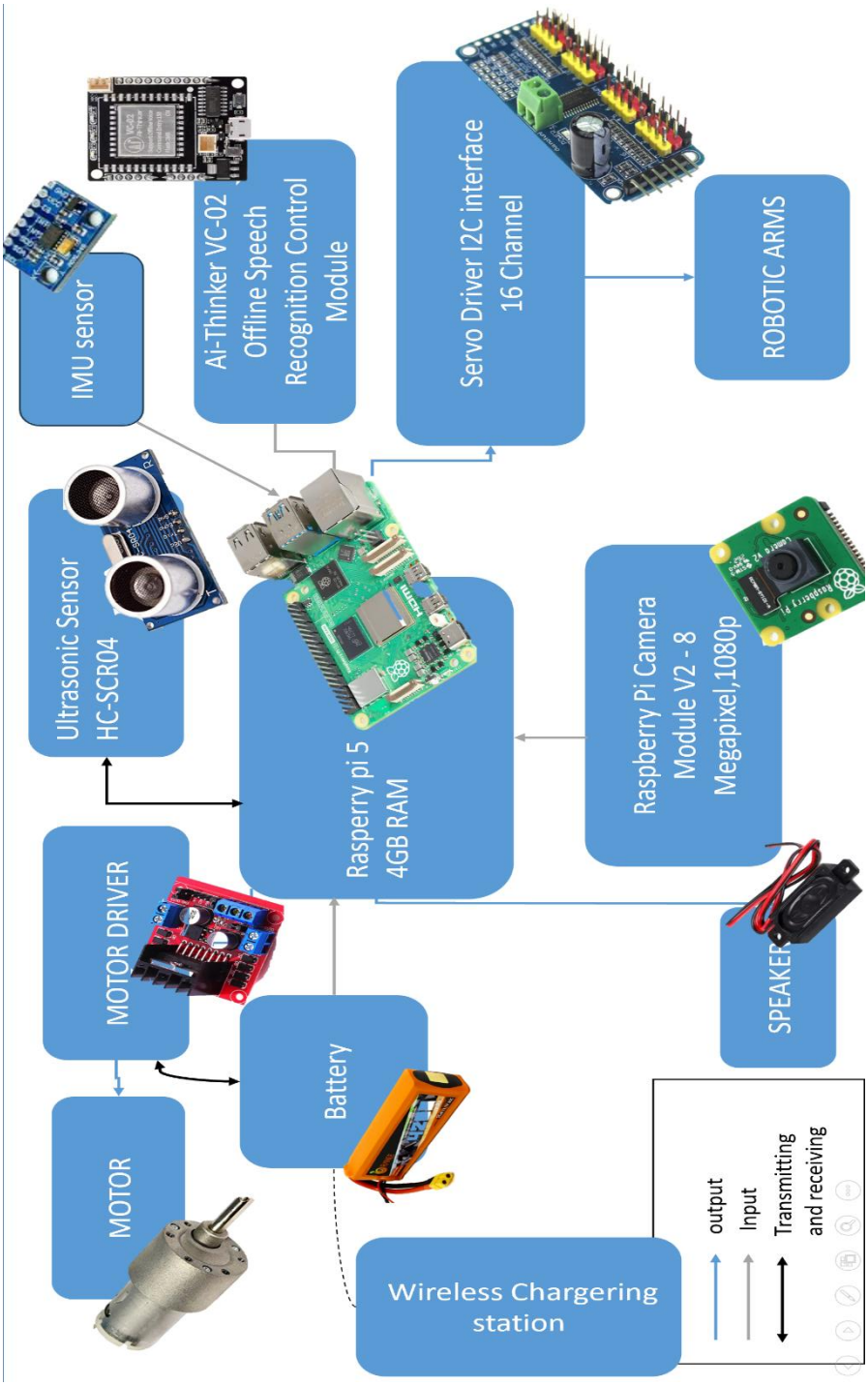
S.No	Component	Description / Specification	Required
1	MG995 Servo Motor	Torque: 10 kg.cm, used for base and elbow joints	4
2	SG90 Servo Motor	Torque: 1.8 kg.cm, used for wrist joint and gripper	8
3	Laser-Cut Acrylic Sheets	Structural material for the robot's frame and joints	1
4	Raspberry Pi 5	Central controller	1
5	Raspberry Pi Camera Noir Module 3	For vision-based applications	1
6	AI Think Voice Recognition Module	Enables voice command functionality	1

7	Ultrasonic Sensor	Assists in distance measurement	1
8	L298N Motor Driver	Controls motor movement	
9	16-bit Servo Driver	Manages multiple servo motors	1
10	200 RPM Motors	Provides robot mobility and speed	1
11	LiPo Battery	Powers the robot system	1
12	3S BMS Module (12.6V, 25A)	Manages charging and discharging of 18650 batteries	1
13	18650 Batteries	Rechargeable cells used in battery pack	1
14	JST / XT60 Connectors	Secure wiring for battery connections	1
15	12V-5A Power Adapter	Power supply for the charging station	1
16	XL4015 DC-DC Step-down Converter	With voltmeter; regulates voltage	1
17	Copper Strips / Tape	Used for contact-based charging	1
18	Schottky Diode (1N5822, 10A)	Prevents reverse current	1
19	10A Fuse	Provides overcurrent protection	1
20	12V Charging Indicator LED + Resistor	Visual indication of charging status	1
21	DC Barrel Jack Connector	Connects adapter to charging circuit	1

BLOCK DIAGRAM

\

3.3CAD



MODELING

The robot is designed to accommodate all essential components efficiently while ensuring proper ventilation for optimal performance and heat management.

Fig [11] : Robot CAD model front view

Fig [12] : Robot CAD model side view

This DXF file is prepared specifically for laser cutting the acrylic sheet components required for assembling the robot's structure.

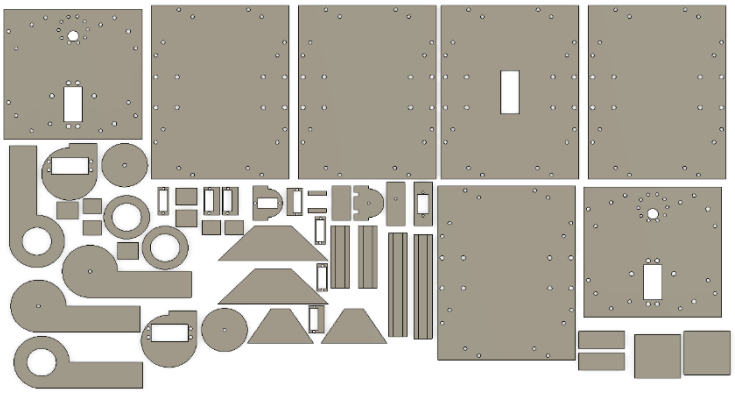


Fig [13] : DXF file of robot

The charging station STL file will include a base platform with mounting points for charging contacts, a central alignment slot for the robot, and space for copper strips or tape to ensure efficient contact-based charging.

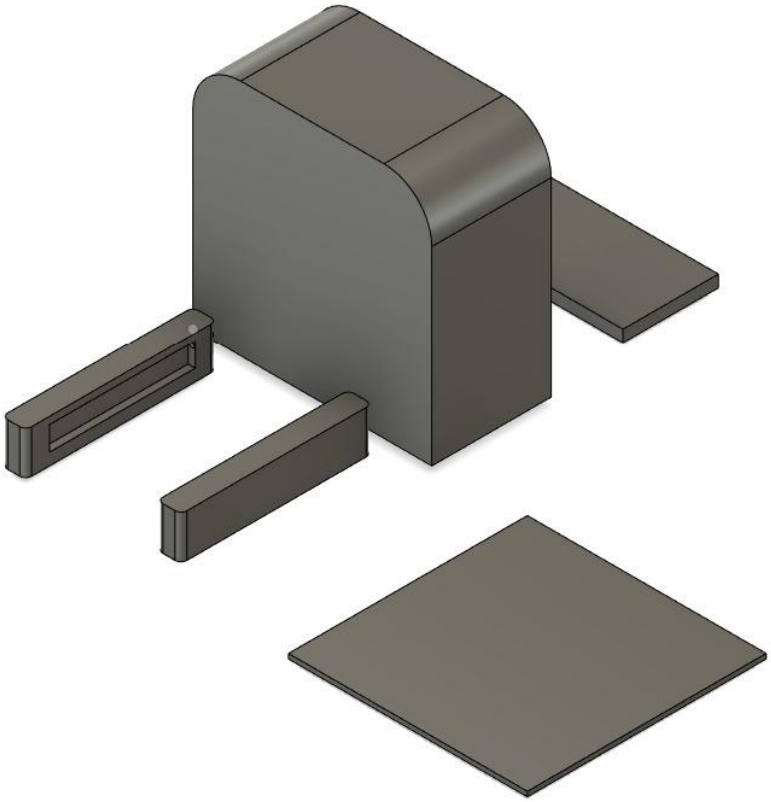


Fig [14] Charging station STLE file

3.4 ALGORITHRIM SELECTION

3.4.1 OBJECT AND FACE DETECTION

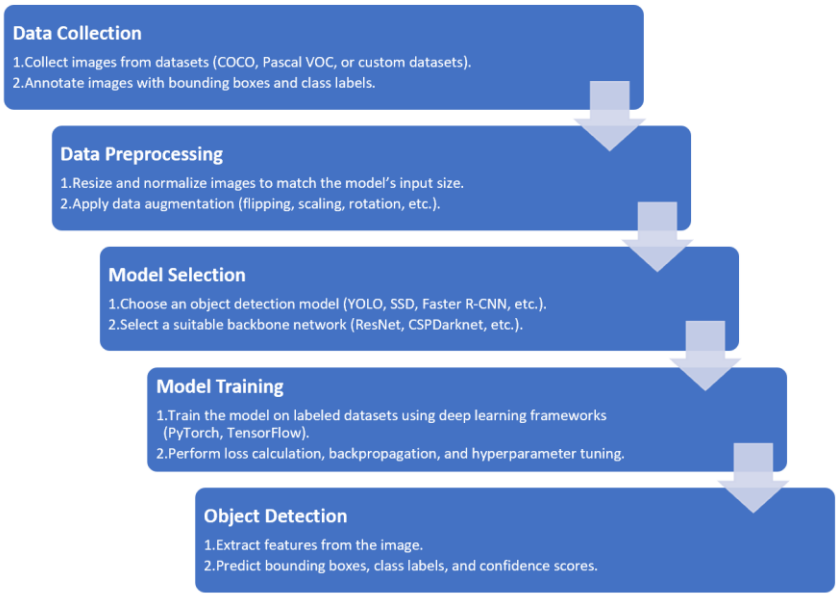
Object detection models are generally classified into single-stage and two-stage detectors. Single-stage models such as YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), and EfficientDet perform detection in one pass, offering high speed and lower latency, which makes them highly suitable for real-time applications like robotics, autonomous drones, and CCTV surveillance. On the other hand, two-stage models like Faster R-CNN, Mask R-CNN, and R-FCN first generate region proposals and then classify them, achieving higher accuracy at the cost of slower processing. These are ideal for applications requiring precision, such as medical imaging, satellite imagery, and autonomous driving.

Among single-stage detectors, YOLO has become extremely popular due to its impressive balance of speed and accuracy. It has evolved from YOLOv1 to YOLOv8, with each version introducing

architectural improvements, better backbone networks, and support for advanced tasks. YOLOv8, developed by Ultralytics, is particularly versatile—it supports not only object detection, but also image classification, segmentation, and pose estimation. Within YOLOv8, the YOLOv8s (small) variant offers the best trade-off between real-time performance and detection accuracy, making it ideal for deployment in embedded systems, low-power robotics, and mobile AI platforms.

In addition to general object detection, face detection is a specialized task that has seen remarkable performance gains using both one-stage and two-stage models. YOLO models, especially YOLOv5 and YOLOv8, can be fine-tuned for real-time face detection, achieving excellent results in applications such as security systems, attendance tracking, and human-robot interaction. For more advanced applications requiring facial landmark detection or emotion recognition, specialized models like MTCNN, RetinaFace, or BlazeFace can be used, often as extensions or complementary networks to detection models.

In summary, for real-time systems like personal assistive robots, mobile surveillance units, or interactive kiosks, YOLOv8s provides a robust solution for both object and face detection, balancing speed and accuracy while maintaining flexibility across multiple vision tasks. For precision-critical use cases, integrating or switching to two-stage models can provide the extra accuracy required, especially when coupled with face-specific detection frameworks.



YOLOv8 is chosen for its compact architecture and rapid inference speed, making it ideal for deployment in embedded and resource-constrained environments. Its small variant, YOLOv8s, offers a balanced trade-off between accuracy and performance, enabling efficient real-time object and face detection. With support for multiple tasks like classification, segmentation, and pose estimation, YOLOv8 is highly versatile. The model's streamlined design allows for quick deployment on devices like Raspberry Pi, NVIDIA Jetson, or mobile processors. Its speed ensures seamless integration into robotics, where timely responses are critical. Furthermore, YOLOv8's open-source ecosystem and continuous updates by Ultralytics make it developer-friendly and production-ready.

3.4.2 A* ALGOTHRIM

This project implements the A* (A-star) algorithm for efficient path planning and navigation. A* is an informed search algorithm that combines the benefits of Dijkstra's algorithm and the Greedy Best-First Search. It evaluates paths using a cost function:

$$f(n) = g(n) + h(n)$$

where g(n) represents the actual cost from the start node to the current node, and h(n) is the heuristic estimate of the cost from the current node to the goal. By balancing both cost and heuristic estimation, A* ensures an optimal and efficient path while avoiding obstacles, making it ideal for real-time robotic navigation.

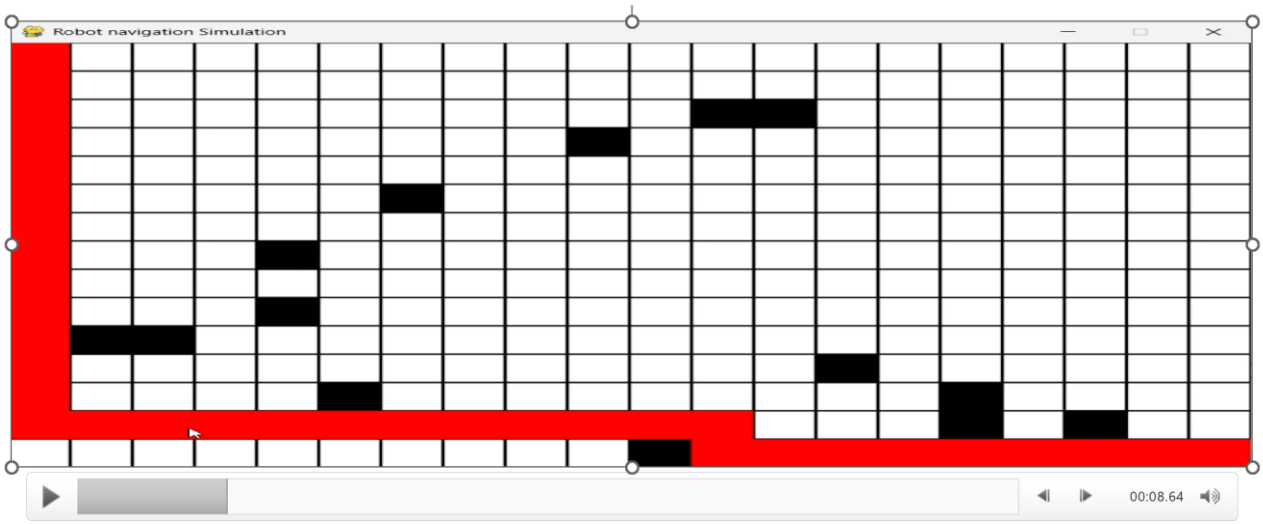


FIG [15]- A* ALGOTHRIM IN NUMP PI

CHAPTER 4

PROGRAMMING

4.1 IMAGE CAPTURING

```
import cv2
import os
from datetime import datetime
from picamera2 import Picamera2
import time

# Change this to the name of the person you're photographing
PERSON_NAME = "sameer"

def create_folder(name):
    dataset_folder = "dataset"
    if not os.path.exists(dataset_folder):
        os.makedirs(dataset_folder)

    person_folder = os.path.join(dataset_folder, name)
    if not os.path.exists(person_folder):
        os.makedirs(person_folder)
    return person_folder

def capture_photos(name):
    folder = create_folder(name)

    # Initialize the camera
    picam2 = Picamera2()
    picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size":
(640, 480)}))
    picam2.start()
```

```
    # Allow camera to warm up
    time.sleep(2)

    photo_count = 0

    print(f"Taking photos for {name}. Press SPACE to capture, 'q' to quit.")

    while True:
        # Capture frame from Pi Camera
        frame = picam2.capture_array()

        # Display the frame
        cv2.imshow('Capture', frame)

        key = cv2.waitKey(1) & 0xFF

        if key == ord(' '):  # Space key
            photo_count += 1
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = f"{name}_{timestamp}.jpg"
            filepath = os.path.join(folder, filename)
            cv2.imwrite(filepath, frame)
            print(f"Photo {photo_count} saved: {filepath}")

        elif key == ord('q'):  # Q key
            break

    # Clean up
    cv2.destroyAllWindows()
    picam2.stop()
    print(f"Photo capture completed. {photo_count} photos saved for {name}.")

if __name__ == "__main__":
```

capture_photos(PERSON_NAME)

Below is a detailed line-by-line explanation of your Raspberry Pi face dataset collection script using the Pi Camera

```
import cv2
import os
from datetime import datetime
from picamera2 import Picamera2
import time
```

Explanation:

- cv2: Used for image display and saving using OpenCV.
- os: Facilitates folder and path handling.
- datetime: Used for timestamping each photo file.
- Picamera2: Allows access to the Raspberry Pi Camera Module 3.
- time: Provides functions to pause or wait during camera startup.

```
PERSON_NAME = "sameer"
```

Explanation:

This variable sets the name of the person being photographed, which is later used to create a dedicated folder for saving that person's images.

```
def create_folder(name):
    dataset_folder = "dataset"
    if not os.path.exists(dataset_folder):
        os.makedirs(dataset_folder)

    person_folder = os.path.join(dataset_folder, name)
    if not os.path.exists(person_folder):
        os.makedirs(person_folder)
    return person_folder
```

Explanation:

- The function create_folder(name) creates a directory structure to store images.
- It first checks if a folder named dataset exists; if not, it creates one.
- Then, it creates (or verifies the existence of) a subfolder within dataset that is named after the person.

- Finally, it returns the path to the person's folder.

```
def capture_photos(name):
    folder = create_folder(name)
```

Explanation:

This line calls the create_folder function to create and retrieve a folder where the captured photos will be saved.

```
    picam2 = Picamera2()
    picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size": (640, 480)}))
    picam2.start()
```

Explanation:

- The Pi Camera (Picamera2) is initialized.
- The camera is configured to capture images in the 'XRGB8888' format at a resolution of 640x480.
- The camera is then started to begin capturing video frames.

```
    time.sleep(2)
```

Explanation:

This pauses the program for 2 seconds, allowing the camera to warm up before capturing images.

```
    photo_count = 0
    print(f"Taking photos for {name}. Press SPACE to capture, 'q' to quit.")
```

Explanation:

- The variable photo_count is initialized to zero to keep track of the number of photos taken.
- A message is printed to inform the user how to capture images (press SPACE) and how to quit (press 'q').

```
    while True:
        frame = picam2.capture_array()
```

Explanation:

- This begins an infinite loop where video frames are continuously captured from the camera using picam2.capture_array().

```
        cv2.imshow('Capture', frame)
```

Explanation:

- The current frame is displayed in a window titled "Capture" using OpenCV.

```
key = cv2.waitKey(1) & 0xFF
```

Explanation:

- The program waits for 1 millisecond for any key press and captures the pressed key (if any).

```
if key == ord(' '):  # Space key
    photo_count += 1
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{name}_{timestamp}.jpg"
    filepath = os.path.join(folder, filename)
    cv2.imwrite(filepath, frame)
    print(f"Photo {photo_count} saved: {filepath}")
```

Explanation:

- If the Space key is pressed:
  - The photo counter is incremented.
  - A timestamp is generated using the current date and time.
  - A filename is created combining the person's name and the timestamp.
  - The full path for the image is created by joining the folder path with the filename.
  - The current frame is saved as a JPEG file at the specified path.
  - A message confirming the photo is printed.

```
elif key == ord('q'):  # Q key
    break
```

Explanation:

- If the 'q' key is pressed, the loop is broken, and the script proceeds to exit.

```
cv2.destroyAllWindows()
picam2.stop()
print(f"Photo capture completed. {photo_count} photos saved for {name}.")
```

Explanation:

- After exiting the loop, all OpenCV windows are closed.
- The camera is stopped to free up resources.
- A message is printed indicating that photo capture is complete and shows the total number of photos saved.

This completes the explanation of your face dataset collection script without using any emojis. Let me know if you need additional details.

4.2 MODEL TRAINING

```
import os
from imutils import paths
import face_recognition
import pickle
import cv2

print("[INFO] start processing faces...")
imagePaths = list(paths.list_images("dataset"))
knownEncodings = []
knownNames = []

for (i, imagePath) in enumerate(imagePaths):
    print(f"[INFO] processing image {i + 1}/{len(imagePaths)}")
    name = imagePath.split(os.path.sep)[-2]

    image = cv2.imread(imagePath)
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    boxes = face_recognition.face_locations(rgb, model="hog")
    encodings = face_recognition.face_encodings(rgb, boxes)

    for encoding in encodings:
        knownEncodings.append(encoding)
        knownNames.append(name)

print("[INFO] serializing encodings...")
data = {"encodings": knownEncodings, "names": knownNames}
```

```
with open("encodings.pickle", "wb") as f:
    f.write(pickle.dumps(data))
print("[INFO] Training complete. Encodings saved to 'encodings.pickle”)
```

Below is a detailed line-by-line breakdown of your face encoding script

Library Imports

```
import os
from imutils import paths
import face_recognition
import pickle
import cv2
```

- os: Helps in file and folder path manipulation.
- imutils.paths: Used to list all image file paths from a directory.
- face_recognition: A Python library for face detection and recognition.
- pickle: Used for saving (serializing) Python objects to a file.
- cv2: OpenCV for image reading and conversion.

Start Message

```
print("[INFO] start processing faces...")
```

- Prints a message indicating that the face processing is starting.

Load All Image Paths

```
imagePaths = list(paths.list_images("dataset"))
knownEncodings = []
knownNames = []
```

- paths.list_images("dataset") generates a list of all image file paths found in the "dataset" folder.
- knownEncodings: An empty list that will store the face encodings.

- knownNames: An empty list that will store the corresponding names for each encoding.

Process Each Image

```
for (i, imagePath) in enumerate(imagePaths):
    print(f"[INFO] processing image {i + 1}/{len(imagePaths)}")
```

- Iterates over each image path using a loop.
- Prints progress information, indicating which image out of the total is currently being processed.

Extract Person's Name from Path

```
name = imagePath.split(os.path.sep)[-2]
```

- Splits the image path by the directory separator and extracts the second-to-last element as the person's name (assuming that the folder name represents the person's name).

Read and Convert Image

```
image = cv2.imread(imagePath)
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

- Reads the image file using OpenCV.
- Converts the image from BGR color space (OpenCV’s default) to RGB color space required by the face_recognition library.

Detect Faces and Compute Encodings

```
boxes = face_recognition.face_locations(rgb, model="hog")
encodings = face_recognition.face_encodings(rgb, boxes)
```

- Detects face locations in the image using the HOG model, which is efficient for CPU-based systems.
- Computes a 128-dimension encoding for each detected face, representing its unique features.

Store Encodings

```
for encoding in encodings:
```

knownEncodings.append(encoding)

knownNames.append(name)

- For each detected face encoding, the encoding is appended to the knownEncodings list.
- The corresponding name (extracted from the folder path) is appended to the knownNames list.

Save Encodings to File

print("[INFO] serializing encodings...")

data = {"encodings": knownEncodings, "names": knownNames}

with open("encodings.pickle", "wb") as f:

f.write(pickle.dumps(data))

- Prints a message indicating that the serializing (saving) process is starting.
- Creates a dictionary containing the face encodings and their names.
- Serializes the dictionary using pickle and writes it to a file named "encodings.pickle".

Done Message

print("[INFO] Training complete. Encodings saved to 'encodings.pickle'")

- Prints a confirmation message stating that the face encoding process is complete and that the encodings have been saved.

Result: After running the script, you will have a file named "encodings.pickle" that contains the facial data (encodings and corresponding names) for your dataset. This file can later be loaded for performing face recognition in real time.

4.3 FACE RECOGNITION

import face_recognition

import cv2

import numpy as np

```
from picamera2 import Picamera2

import time

import pickle

Load pre-trained face encodings

print("[INFO] loading encodings...")

with open("encodings.pickle", "rb") as f:

data = pickle.loads(f.read())

known_face_encodings = data["encodings"]

known_face_names = data["names"]

Initialize the camera

picam2 = Picamera2()

picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size": (1920, 1080)}))

picam2.start()

Initialize our variables

cv_scaler = 4 # this has to be a whole number

face_locations = []

face_encodings = []

face_names = []

frame_count = 0

start_time = time.time()

fps = 0

def process_frame(frame):

global face_locations, face_encodings, face_names

# Resize the frame using cv_scaler to increase performance (less pixels processed, less time spent)

resized_frame = cv2.resize(frame, (0, 0), fx=(1/cv_scaler), fy=(1/cv_scaler))

# Convert the image from BGR to RGB colour space, the facial recognition library uses RGB, OpenCV uses BGR

rgb_resized_frame = cv2.cvtColor(resized_frame, cv2.COLOR_BGR2RGB)

# Find all the faces and face encodings in the current frame of video

face_locations = face_recognition.face_locations(rgb_resized_frame)
```

```
face_encodings = face_recognition.face_encodings(rgb_resized_frame, face_locations,
model='large')
face_names = []
for face_encoding in face_encodings:
    # See if the face is a match for the known face(s)
    matches = face_recognition.compare_faces(known_face_encodings, face_encoding)
    name = "Unknown
    # Use the known face with the smallest distance to the new face
    face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
    best_match_index = np.argmin(face_distances)
    if matches[best_match_index]:
        name = known_face_names[best_match_index]
    face_names.append(name)
return frame
def draw_results(frame):
# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was scaled
    top *= cv_scaler
    right *= cv_scaler
    bottom *= cv_scaler
    left *= cv_scaler
    # Draw a box around the face
    cv2.rectangle(frame, (left, top), (right, bottom), (244, 42, 3), 3)
    # Draw a label with a name below the face
    cv2.rectangle(frame, (left -3, top - 35), (right+3, top), (244, 42, 3), cv2.FILLED)
    font = cv2.FONT_HERSHEY_DUPLEX
    cv2.putText(frame, name, (left + 6, top - 6), font, 1.0, (255, 255, 255), 1)
return frame
def calculate_fps():
```

```
global frame_count, start_time, fps
frame_count += 1
elapsed_time = time.time() - start_time
if elapsed_time > 1:
    fps = frame_count / elapsed_time
    frame_count = 0
    start_time = time.time()
return fps
while True:
# Capture a frame from camera
frame = picam2.capture_array()
# Process the frame with the function
processed_frame = process_frame(frame)
# Get the text and boxes to be drawn based on the processed frame
display_frame = draw_results(processed_frame)
# Calculate and update FPS
current_fps = calculate_fps()
# Attach FPS counter to the text and boxes
cv2.putText(display_frame, f"FPS: {current_fps:.1f}", (display_frame.shape[1] - 150, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Display everything over the video feed.
cv2.imshow('Video', display_frame)
# Break the loop and stop the script if 'q' is pressed
if cv2.waitKey(1) == ord("q"):
    break
By breaking the loop we run this code here which closes everything
cv2.destroyAllWindows()
picam2.stop()
```

Below is a detailed line-by-line explanation of your real-time face recognition script that uses the Raspberry Pi camera and the face_recognition library, presented without any emojis.

Importing Required Libraries

```
import face_recognition

import cv2

import numpy as np

from picamera2 import Picamera2

import time

import pickle
```

- These libraries serve the following purposes:
    - **face_recognition**: Provides functions for detecting and recognizing faces.
    - **cv2 (OpenCV)**: Used for image capture, display, and drawing annotations.
    - **numpy**: Supports efficient array operations necessary for image processing.
    - **Picamera2**: Controls the Raspberry Pi Camera (Noir Module 3).
    - **time**: Provides time functions, such as pausing execution or measuring elapsed time.
    - **pickle**: Handles serialization and deserialization of Python objects (used for loading saved face encodings).

Load Known Encodings

```
print("[INFO] loading encodings...")

with open("encodings.pickle", "rb") as f:

    data = pickle.loads(f.read())

known_face_encodings = data["encodings"]

known_face_names = data["names"]
```

- A message is printed to indicate that the script is loading face encodings.

- The file "encodings.pickle" is opened in binary read mode, and its contents are loaded using pickle.
- The variable known_face_encodings is assigned the list of facial feature vectors, and known_face_names is assigned the corresponding names. These are the “trained” data for face recognition.

Initialize Pi Camera

```
picam2 = Picamera2()

picam2.configure(picam2.create_preview_configuration(main={"format": 'XRGB8888', "size": (1920, 1080)}))

picam2.start()
```

- A new instance of the Pi Camera is created.
- The camera is configured for preview mode with a resolution of 1920x1080 and an image format of XRGB8888. This high resolution improves detection accuracy.
- The camera is started to begin capturing frames.

Initialize Tracking Variables

```
cv_scaler = 4  # Scale factor to downsize frame for faster processing

face_locations = []

face_encodings = []

face_names = []

frame_count = 0

start_time = time.time()

fps = 0
```

- cv_scaler is set to 4, which downsizes each frame to increase processing speed by reducing the number of pixels processed.
- Lists for face_locations, face_encodings, and face_names are initialized to store data for each frame.

- Variables frame_count, start_time, and fps are initialized for measuring the frames per second (FPS).

Function to Process Each Frame

def process_frame(frame):

This function is responsible for:

1. Downsizing the frame to improve processing speed.
2. Converting the frame from BGR (used by OpenCV) to RGB (required by the face_recognition library).
3. Detecting faces and computing their encodings.
4. Comparing detected encodings with known face encodings.
5. Identifying each face by name or marking it as "Unknown."

Function to Draw Results on the Frame

def draw_results(frame)

- This function draws the detected results on the frame by:
    - Drawing bounding boxes around each detected face.
    - Displaying the corresponding name for each recognized face.
    - Scaling the face location coordinates back up to the original frame size using the cv_scaler.

Function to Calculate Frame Per Second (FPS)

def calculate_fps():

- This function tracks the number of frames processed over a period of time and calculates the frames per second (FPS) for monitoring the performance of the face recognition system.

Main Loop: Real-Time Detection

while True:

```
frame = picam2.capture_array()
```

processed_frame = process_frame(frame)

display_frame = draw_results(processed_frame)

current_fps = calculate_fps()

- The script enters an infinite loop, where:
    - A frame is continuously captured from the Pi Camera.
    - The frame is processed to detect and recognize faces.
    - The processed results (bounding boxes, names, etc.) are drawn on the frame.
    - The FPS is calculated based on the number of frames processed per second.

Display Frame + Quit on 'Q'

cv2.putText(display_frame, f"FPS: {current_fps:.1f}", ...)

cv2.imshow('Video', display_frame)

if cv2.waitKey(1) == ord("q"):

break

- The current FPS is overlaid on the display frame.
- The annotated frame is shown in a window titled "Video."
- The loop continuously checks for a key press, and if the 'q' key is pressed, the loop terminates.

Clean-Up

cv2.destroyAllWindows()

picam2.stop()

- After the loop exits, all OpenCV windows are closed.
- The Pi Camera is stopped to release resources properly.

Summary

This script utilizes pre-trained face encodings to recognize faces in real-time. It captures live video from the Raspberry Pi Camera, processes each frame to detect and identify faces, and displays the results along with an FPS counter. The script is structured to provide a real-time face recognition system that continuously updates until manually stopped by pressing the 'q' key.

Let me know if further modifications or explanations are needed.

4.4 OBJECT DETECTION

from ultralytics import YOLO import cv2 from picamera2 import Picamera2 import time

Load model (use YOLOv8n or YOLOv8s for fast inference)

model = YOLO('yolov8n.pt')

Initialize Pi Camera

picam2 = Picamera2() picam2.preview_configuration.main.size = (640, 480)
picam2.preview_configuration.main.format = "RGB888" picam2.configure("preview")
picam2.start()

time.sleep(1)

while True: frame = picam2.capture_array()

Run YOLOv8 detection

results = model(frame, verbose=False)[0]

Draw results on frame

annotated_frame = results.plot()

cv2.imshow("YOLOv8 Live Detection", annotated_frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
  break
cv2.destroyAllWindows()
Below is a detailed, plain-text explanation of the provided code:

1. Import Libraries
2. from ultralytics import YOLO

3. import cv2
4. from picamera2 import Picamera2
5. import time
    - The code imports the YOLO model interface from the ultralytics package.
    - cv2 (OpenCV) is imported for image display and processing.
    - Picamera2 is used to access and control the Raspberry Pi Camera.
    - time provides time functions such as sleep.
6. Load the YOLO Model
7. model = YOLO('yolov8n.pt')
    - This line loads a pre-trained YOLOv8 "nano" model from the file 'yolov8n.pt'.
    - The YOLOv8n model is chosen for its speed and efficiency, suitable for real-time inference on resource-constrained devices.
8. Initialize the Pi Camera
9. picam2 = Picamera2()
10. picam2.preview_configuration.main.size = (640, 480)
11. picam2.preview_configuration.main.format = "RGB888"
12. picam2.configure("preview")
13. picam2.start()
    - An instance of the Pi Camera is created.
    - The camera's preview configuration is set to a resolution of 640x480 pixels.
    - The image format is configured as "RGB888" (which means it outputs 8-bit each for red, green, and blue channels).
    - The camera is configured for preview mode and then started, which begins capturing frames.
14. Pause for Camera Warm-Up

15. time.sleep(1)

    - The program pauses for 1 second to allow the camera sensor to stabilize before processing frames.

16. Main Loop for Live Object Detection

17. while True:

18. frame = picam2.capture_array()

19. # Run YOLOv8 detection

20. results = model(frame, verbose=False)[0]

21. # Draw results on frame

22. annotated_frame = results.plot()

23. cv2.imshow("YOLOv8 Live Detection", annotated_frame)

24. if cv2.waitKey(1) & 0xFF == ord('q'):

25. break

    - The loop continuously captures frames from the camera using picam2.capture_array().
    - For each captured frame, the YOLO model is used to perform object detection. The call to model(frame, verbose=False)[0] returns the detection results for the frame.
    - The results.plot() method draws the detection results (such as bounding boxes and labels) onto the frame.
    - The annotated frame is then displayed in a window titled "YOLOv8 Live Detection".
    - The loop checks every frame to see if the 'q' key is pressed. If detected, the loop is exited.

26. Clean Up Resources

27. cv2.destroyAllWindows()

    - After the loop ends (when 'q' is pressed), all OpenCV windows are closed.

- Finally, the camera is stopped (code not shown in the snippet after the loop, but it is implied as a cleanup step).

This script captures a live video stream from the Raspberry Pi Camera, applies YOLOv8 for object detection on each frame, and displays the annotated live feed until the user stops the program by pressing 'q'.

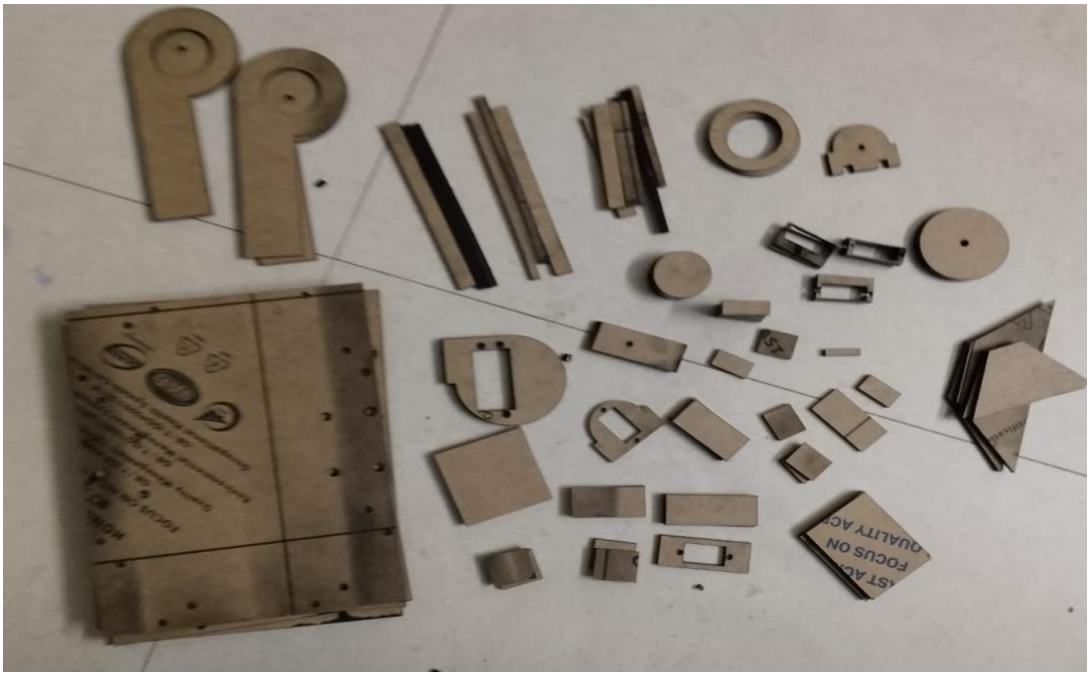
CHAPTER 5

RESULT

5.1 HARDWARE ASSEMBLY

FIG [16]-GIPPER 3D PRINTED

FIG[17]-CHARGING STATION 3D PRINTED



FIG[18] ARYCLIC SHEETS

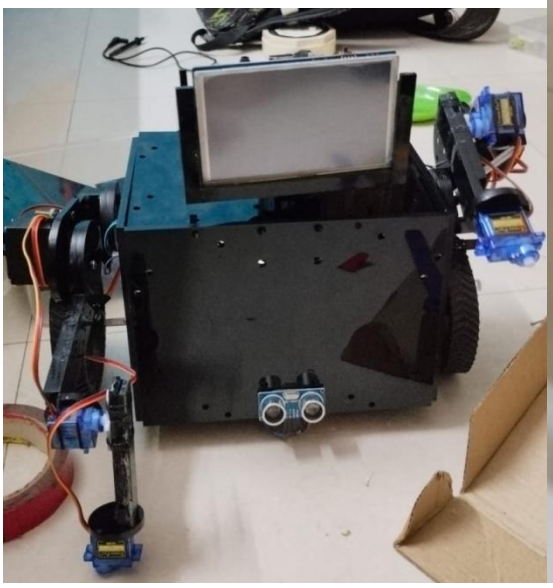


FIG [19]-FULLY ASSEMBLED

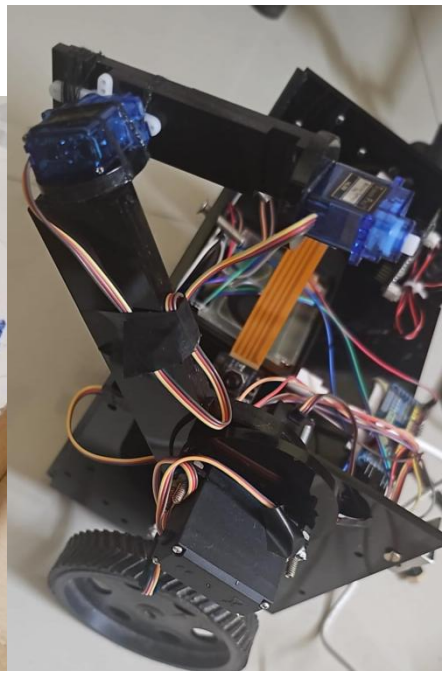


FIG [20]-ROBOTIC ARM

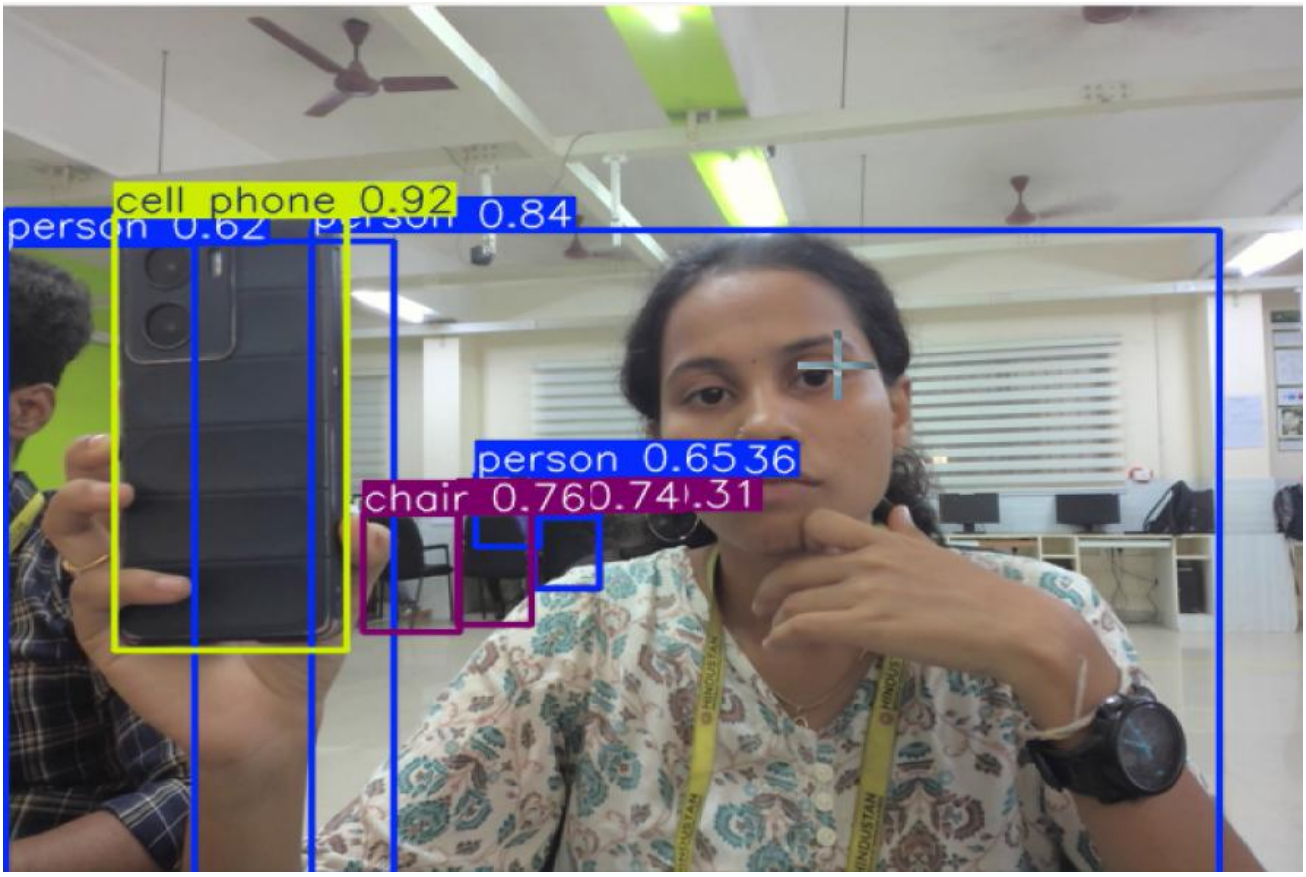


FIG [21]OBJECT DETETCTION

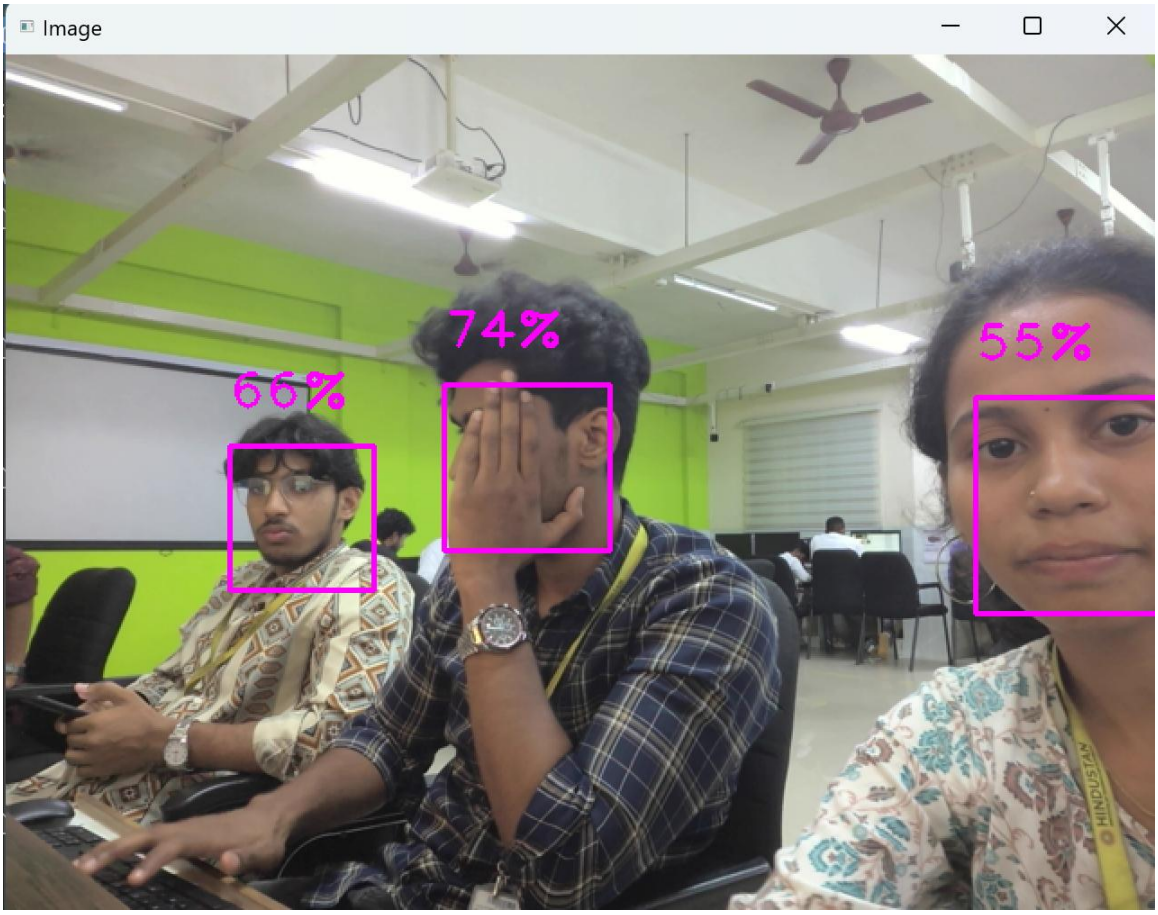


FIG [22] FACE DETECTION

CHAPTER 6

CONCLUSION

The development of the *Personal Assistive Robot* represents a significant step toward integrating intelligent systems into daily life for enhanced convenience, interaction, and functionality. By combining computer vision, voice recognition, robotic manipulation, and autonomous mobility, the robot demonstrates a multidisciplinary approach to solving real-world problems in assistive technology.

The project successfully implemented two robust path planning algorithms—A* and RRT—each evaluated for their suitability in navigation within constrained indoor environments. YOLOv8 was employed for object and face detection, ensuring real-time visual perception. Voice command functionality enabled intuitive user interaction, while the dual 5-DOF robotic arms extended the robot's capabilities for pick-and-place operations.

From a design perspective, the robot was efficiently constructed using laser-cut acrylic components and 3D-printed modules, ensuring both structural integrity and customization. The integration of the Raspberry Pi 5 with peripheral modules like the Noir camera, voice recognition module, and ultrasonic sensors enabled a compact yet powerful control system.

In conclusion, this project not only achieved its technical objectives but also laid the foundation for future improvements, such as enhanced emotion recognition, obstacle avoidance with dynamic path updates, and cloud connectivity for remote control. The assistive robot showcases the potential of low-cost, intelligent robotics in improving quality of life across a wide range of applications—from smart homes to healthcare and beyond.

CHAPTER 7

REFERENCES

[1] Ishiguro, Hiroshi, Tetsuo Ono, Michita Imai, and Takayuki Kanda.Development of an Interactive Humanoid Robot 'Robovie'—An Interdisciplinary Approach.,2001

[2] Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn.A Survey of Socially Interactive Robots.,2003

[3]Ratsamee, Photchara, Yasushi Mae, Kazuto Kamiyama, Mitsuhiro Horade, Masaru Kojima, and Tatsuo Arai,Social Interactive Robot Navigation Based on Human Intention Analysis from Face Orientation and Human Path Prediction. ,2015

[4] Mišeikis, Justinas, Pietro Caroni, Patricia Duchamp, Alina Gasser, Rastislav Marko, Nelija Mišeikienė, Frederik Zwilling, Charles de Castelbajac, Lucas Eicher, Michael Früh, and Hansruedi Früh,Lio-A Personal Robot Assistant for Human-Robot Interaction and Care Applications.,2020

[5] Mariagrazia Costanzo, Rossana Smeriglio, and Santo Di Nuovo,” New Technologies and Assistive Robotics for Elderly: A Review on Psychological Variables”,2024

[6] Duerstock, Bradley S,Report on the Use of Assistive Robotics to Aid Persons with Disabilities,2024